Project Report

Ben Friedl

- Run-Length Encoding
- LZ4

Note: I sometimes refer to byte 'stream's in this write-up, but these are all slices/byte arrays in my implementation

Lossless data compression allows taking advantage of patterns in data in order to store/transmit data in fewer bytes. A variety of lossless data compression algorithms exist. There are slow algorithms suitible for long-term cold-storage like PAQ [w] or NNCP that provide high compression ratios, but are expensive to compress and decompress. There are also faster algorithms like LZ4 [w] or snappy [w] that are faster to compress and decompress, and can actually improve read/write speeds on modern storage devices (OpenZFS recommends LZ4 and claims "the reduction in IO from LZ4 will typically be a performance win"). In-between algorithms like the still-popular Deflate (used by zlib) [w] or Zstd [w] are suitible for network transfer (on networks that aren't extremely fast) or as a compromise between performance and disk usage.

My project focused on verifying simple and fast compression algorithms, specifically a simple run-length encoder [w] and LZ4. These algorithms are notable for not having any form of entropy-encoding [w] (such as huffman encoding, arithmetic encoding, or tANS). Instead all of the compression comes from identifying repeated sequences of bytes.

Run-Length Encoding

Before I attempted to verify LZ4, I wanted to start with something simpler. The compression format for my run-length encoder is very simple: an encoded file consists of zero or more runs. Each run is two bytes, and the first byte specifies how many times to repeat the second byte. If a byte is repeated more than 255 times than the encoder must store it as two or more runs.

I initially tried to prove a Go implementation of a decompressor for this format by proving a rocq decompressor that transformed bytes -> bytes by directly inducting over bytes, then proving the Go code matched the rocq decompressor. While I was able to prove the rocq byte-wise decompressor matched parsing the bytes then decompressing from an AST, I had difficulty proving the Go code

matched rocq decompressor.

To simplify the proof I proved a weaker specification: if the input is valid then the output of the decompressor is valid. This is a similar specification to the guarantees offered by real-world compression libraries in practice. For performance reasons, decompressors often skip checks not necessary for reliability/security concerns. When given these invalid inputs, the libraires return a byte stream that the program will then treat as valid instead of an error. This isn't an issue in practice since inputs to a compressor are usually from a compressor, which should return a valid compressed stream.[1]

Weakening the specification this way allows proof of the Go code to induct over the AST instead of over bytes by using the fact that the input is a valid compressed stream means the inputs stream is the unparsed form of a valid AST. Doing this requires proving the AST uniquly represents a compressed stream and vise versa. Once these proofs are done there is no longer a need to induct over compressed bytes.

I proved this weaker specification for a Go decompression implementation in (rle.v). This specification could be combined with a proof of 'reasonableness' (returns an error or a length less than the output buffer for all inputs) for a specification of what people expect of a compression library (I did not write such a proof since this isn't a practical compresssion algorithm for most real-world data).

I also proved the correctness (can compress any input bytes to compressed bytes that decompress to the input) of a compressor written in rocq (rle_pure.v#rle_comp).

LZ4

After proving the run-length encoder I began work on proving an LZ4-decompressor (specifically the block format, in order to compress arbitary-sized files an additional frame format must be used). An informal LZ4 block format specification is provided by the author of the format. [] An lz4 compressed block can be thought of as two different operations: literals and matchcopies. Literals represent uncompressed bytes (to append to the output stream) and matchcopies represent a copy from the already decompressed bytes to the end of the output stream. Notably, the length of a matchcopy can exceed the end of the current output which can be interpreted as copying from bytes that will be placed earlier in the matchcopy operation (alternatively it can be thought of as looping back to the start of the copy again). This allows it to function as run length encoding as well as encode

repeated seqences longer than one byte in addition to simply copying existing bytes once.

The format follows a {N*[Lit + MatchCopy] + Lit} pattern where N >= 0. The upper four bits of the first byte of a literal are used to store the literal length and the lower four bits are used to store the length of a subsequent matchcopy. If either lengths are 15 (the max value) additional variable length unsigned integers are added to the respective values and byte stream (although, not immedately in the case of matchcopy). Matchcopies contain a fixed length offset for the start of where to copy from. How long of values to allow is up to implementations, but the spec recommends 4MiB. For full details on LZ4 see the spec I linked earlier.

I wrote a Fixpoint (in lz4_spec.v) over bytes to convert LZ4 compressed streams into an AST. This Fixpoint is extremely ugly in order for rocq to understand that the remaining bytes is always decreasing. I investigated other approaches for this ugly Fixpoint (in part due to later proof difficulties), but it seems other approaches (a summary of which can be found in Well-founded recursion done right (was supprised to see who introduced the talk for this paper [▶] :))) would also be complicated so I left it alone. This specification allows a slight superset of valid LZ4 streams (since it does not reject streams invalid for historical reasons for example). This specification is fine for a decompressor, but if a compressor were to be written against the same spec it should add stronger guranetees.

I tested my specification by compressing a few test cases with the LZ4 C implementation and using OCaml extraction and an LLM-generated OCaml driver for calling the rocq decompression code. Since it correctly decompresses a few different files, I can be confident that any remaining bugs are probably subtle.

I was ultimately unsuccessful at proving a Go implementation of LZ4 because I struggled to prove the parsed AST and unparsed bytes were equivelent. There were two main difficulties in proving this: First, since the matchcopies and literals share information in the first byte there is no clean way to seperate the two. I had to represent a literal followed by a matchcopy and a literal followed by termination as the two nodes in my AST. Then to make things worse, each literal or matchcopy can either have or not have an additional variable length integer resulting in 6 possible branches. Second, I was unable to reason about the bitwise operations effectively. I didn't find any useful lemmas and couldn't make sense of 'bitblast'.

Once that proof is finished proving an unoptimized Go version of LZ4 block decompression should be

a roughly similar amount of work to proving the bytes and AST are equivelent. Said proof/Go function could then be combined with a frame-format and a verified compressor to create a compression library with strong end-to-end correctness guarantees (the input data is always the output data for an arbitrary input).

Overall this project was significantly more difficult than I expected, and I had many false-starts even when verifying the run-length encoder.

---

1. There is the security concern that different decompressors having different decompression behavior could be used to avoid anti-virus scans and the like. The property that different programs will interpret files the same way in the case of malicious inputs is violated by many popular file formats such as ZIP or XML. It is therefore 'reasonable' to expect such software to simply scan against all possible decompression/parsing implementations that the file will be used under. Also proving you reject all illegal streams isn't really that meaningful for this security property unless all other popular implementations do the same.↩